

PriceCompare: Cross-Store Price Comparison with Product Matching and Price Tracking

Midterm Project Report

Submitted by: _____ Roll No.: _____

Course / Programme: _____ Guide: _____

Date: October 2026

Source code: github.com/alizeeshan1234/price-compare-capstone-project | Live: price-compare-alpha-lime.vercel.app

Abstract

Online shoppers in India must open several e-commerce sites to find the cheapest price for a product, and the same product is titled differently on every site. PriceCompare is a web application that accepts one product name, searches six stores (Amazon, Flipkart, Snapdeal, Vijay Sales, Croma and Reliance Digital) in parallel, identifies which listings refer to the same product, and presents the prices side by side with the cheapest highlighted. It records prices daily to show a 30-day trend, and lets users set price-drop alerts that are checked by a scheduled job. The system is built with Python and FastAPI, is covered by 69 automated tests, and is deployed on Vercel. The product-matching algorithm was evaluated on 78 hand-labelled listing pairs and reaches 92.7% precision and 95.0% recall. This report describes the problem, the design, the work completed so far, its evaluation, and the work planned for the remainder of the project.

1. Introduction

Prices for the same electronic product frequently differ by 5–15% between Indian online stores, and the difference changes from day to day. Comparing manually is slow: each store has its own search, its own way of naming products (for example "Apple iPhone 15 (128 GB) – Black" versus "APPLE iPhone 15 (Black, 128 GB)"), and listings for accessories such as cases and chargers clutter the results. Existing comparison sites either cover few stores, rely on paid data feeds, or do not show whether today's price is actually a good one.

The aim of this project is to build a working price-comparison service that solves three technical problems: fetching live data from stores that do not provide public APIs, deciding automatically when two differently worded listings are the same product, and tracking prices over time so that the user can judge a deal.

2. Objectives

1. Search multiple stores concurrently from a single query and tolerate individual store failures.

2. Group listings of the same product across stores while keeping different variants (storage, RAM, Pro/Plus models) apart.
3. Present a side-by-side comparison with the lowest price and the saving highlighted, loading progressively so that fast stores appear first.
4. Record daily prices and show a 30-day trend per product.
5. Allow users to set price-drop alerts that are checked automatically and delivered by email.
6. Measure the accuracy of the matching algorithm on a labelled dataset, and cover the system with automated tests.
7. Deploy the application publicly.

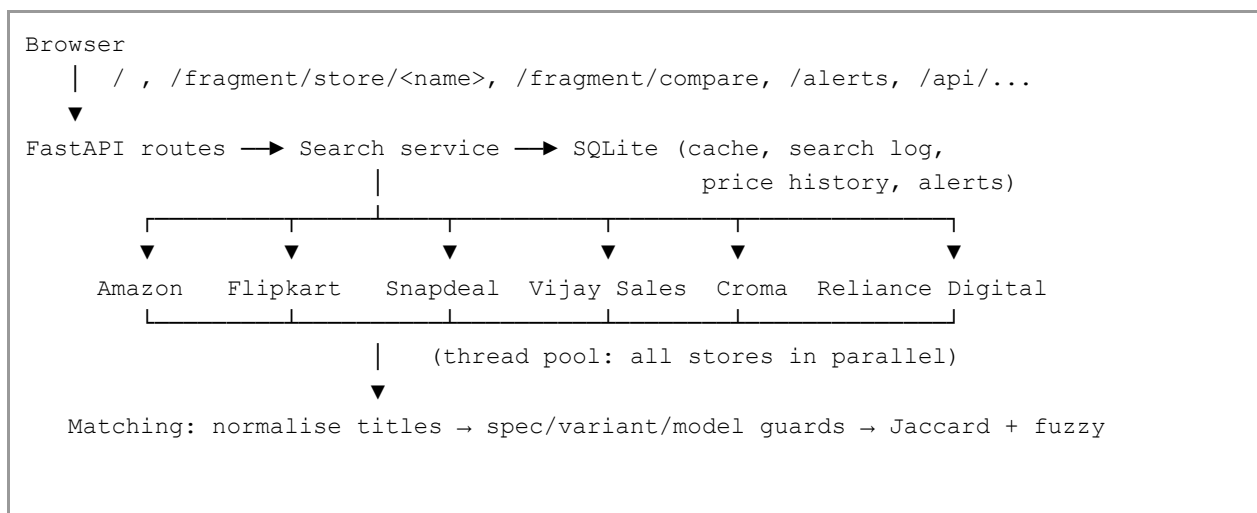
3. Existing Systems

System	Approach	Limitation addressed by this project
Store search pages	One store at a time	No cross-store view; manual comparison.
Google Shopping	Merchant data feeds	Coverage depends on paid feeds; no price history for most Indian stores.
Price-tracker sites (e.g. PriceHistory-type tools)	Track one listing URL	Do not match the same product across stores; user must find each URL.
Browser extensions	Overlay on a product page	Work only once the user is already on a product page.

PriceCompare combines live multi-store search, automatic product matching and price history in one open-source application without relying on commercial data feeds.

4. System Architecture

The application follows a layered design. A FastAPI web layer serves HTML pages and a JSON API. A search service coordinates a cache, the store adapters and the matching module. SQLite stores cached results, a search log, daily price history and alerts.



Module	Responsibility
app/stores/*.py	One adapter per store: builds the search URL, fetches (with retries and optional proxy) and parses listings into a common <code>Offer</code> (store, title, price, URL, image, rating).
app/stores/registry.py	List of stores; runs all adapters concurrently in a thread pool so a search takes as long as the slowest store, not the sum.
app/matching.py	Title normalisation and cross-store grouping (Section 5.2).
app/search.py	Cache lookup, store search, grouping and attaching price history; supports single-store searches for progressive loading.
app/cache.py	SQLite tables: results cache (3-hour TTL), search log, price history, alerts.
app/alerts.py, app/notifier.py	Price-drop checker and email delivery.
app/main.py, templates/	Routes and Jinja2 pages.

5. Implementation

5.1 Store adapters

None of the six stores offer a public API, so each adapter uses the most stable data source available: Amazon and Snapdeal are parsed from server-rendered HTML; Flipkart and Reliance Digital embed the first page of results as JSON inside the page (`window.__INITIAL_STATE__`), which is far more stable than their frequently changing CSS class names; Vijay Sales and Croma are read from the JSON search services their own web pages call. Each adapter detects CAPTCHA or "access denied" pages and reports a clear error instead of returning an empty result. Every adapter has an offline unit test against a saved copy of a real search page, so breakage caused by a store changing its markup is detected immediately. Stores block requests from cloud data-centre IP addresses, so the hosted copy routes fetches through a residential proxy service.

5.2 Product matching

The matcher decides whether two listings are the same product. It is deliberately rule-based so that every decision can be explained.

1. **Normalisation.** Titles are lower-cased and tokenised. Specifications such as "128 GB" are fused into one token (`128gb`), "15.40 cm" becomes `15d40cm` so the number cannot leak into matching, hyphenated model codes ("WH-1000XM5") are kept whole, "Pro+" becomes "pro plus", and marketing filler words are removed.

2. **Guards.** Two listings can only be merged if their specifications do not contradict: storage/RAM, battery, power and weight values must agree (a listing that omits RAM still matches, 8 GB vs 16 GB does not); screen sizes must be within 1.5 inches because stores round cm and inches differently; variant words (Pro, Max, Plus, Ultra, Air, FE...) and model numbers (15 vs 16, Flip 5 vs Flip 6, S24 vs S23) must agree.
3. **Similarity.** Jaccard similarity of the token sets must reach 0.45. A shared model code such as "G502" is accepted on its own. A character-level fuzzy score (rapidfuzz) rescues pairs just under the threshold whose normalised titles are nearly identical strings, which handles spelling and spacing differences like "Fire-Boltt" vs "Fire Bolt".
4. **Clustering and ranking.** Offers are clustered greedily, cheapest first, into one row per product. Accessories (cases, chargers) are detected and sink to the bottom; rows are ranked by how closely they match the query and how many stores carry them.

5.3 Progressive loading

Stores answer at very different speeds (0.4 s to 9 s). Instead of blocking until the slowest store replies, the results page renders immediately with placeholder columns; the browser requests each store separately (`/fragment/store/<name>`) and fills columns as they arrive, with a progress bar. When the last store answers, the browser posts the collected results to `/fragment/compare`, which groups them and returns the comparison table. Fully cached queries are rendered server-side in one step.

5.4 Price history

Every live result is recorded once per listing per day, keyed by a canonical product URL (the ASIN for Amazon, the product id for Flipkart). For each product row the lowest daily price over the last 30 days is drawn as a sparkline, with the 30-day low and a "lowest in N days" flag when today's price is the minimum.

5.5 Price-drop alerts

From any row the user enters an email and a target price. A checker re-searches the watched queries, finds the exact listing (or the same product in another store using the matcher) and, when the price is at or below the target, sends an email through SMTP and marks the alert as triggered. The checker runs from the command line, from the alerts page, or on a daily schedule (Vercel Cron) protected by a secret token. Without SMTP configured, triggers are recorded on the alerts page so the flow can be demonstrated offline.

5.6 Caching, resilience and health

Results are cached for three hours, so repeat searches are instant and stores are not overloaded. A store that fails (blocked, timed out, changed markup) fails alone: its column shows the reason while the other stores render normally. Every search is logged and summarised in a store-health table (success rate, average time, average results over 7 days). A demo mode with simulated stores allows the application to run offline for presentations.

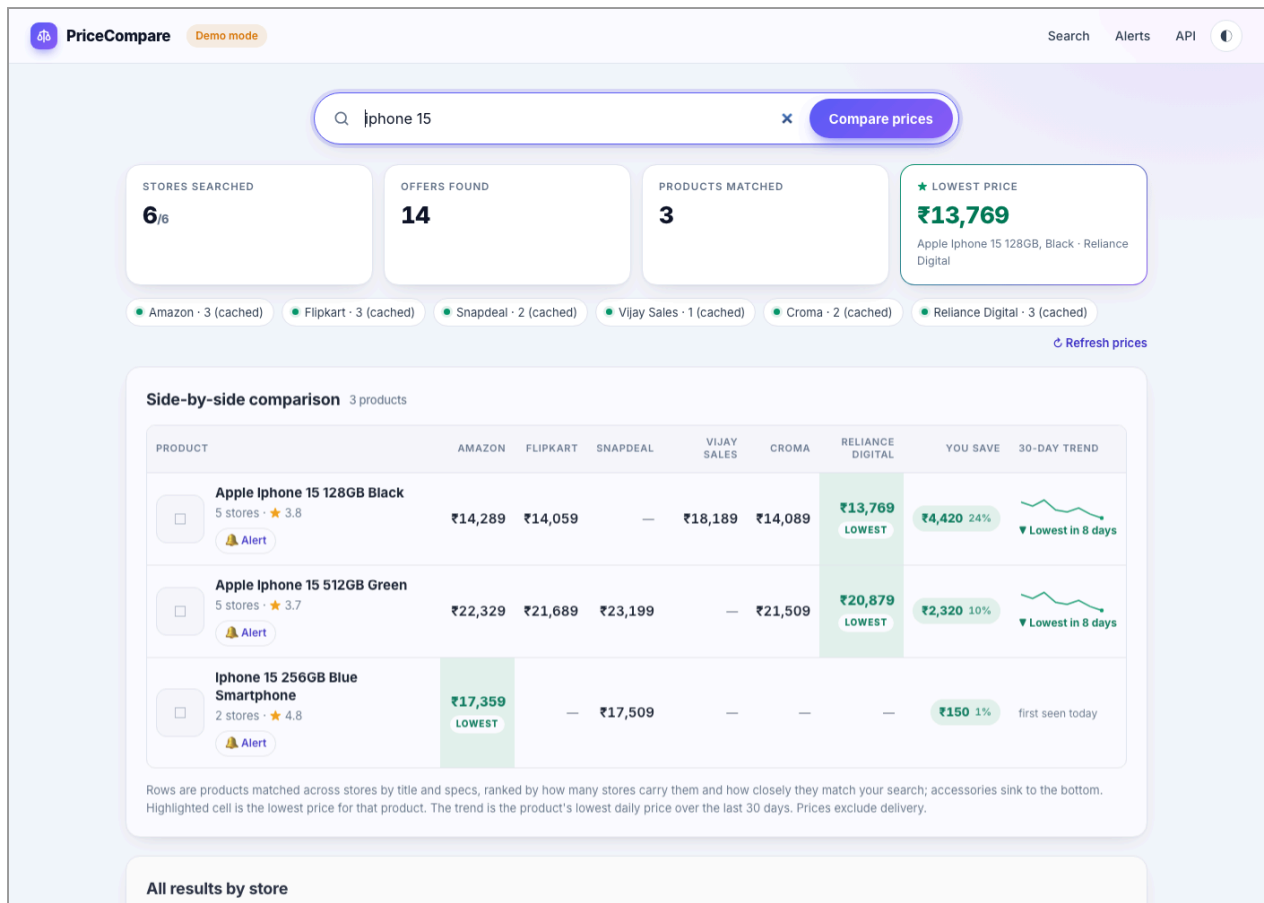


Figure 1: Results page for "iphone 15" (demo mode): summary, store status, side-by-side comparison with lowest price, saving and 30-day trend.

6. Technologies Used

Area	Technology
Language / framework	Python 3, FastAPI, Uvicorn, Jinja2 templates
Data extraction	requests, BeautifulSoup, lxml; JSON parsing of embedded page state and store search services
Matching	Custom tokeniser and rules; rapidfuzz for character-level similarity
Storage	SQLite (results cache, search log, price history, alerts)
Front end	HTML/CSS with a small amount of JavaScript for progressive loading; responsive, dark mode
Testing	pytest with saved page fixtures and simulated stores (69 tests)
Deployment	Vercel (serverless), Docker, GitHub; Vercel Cron for the alert checker

7. Testing and Evaluation

7.1 Automated tests

Test area	What is checked	Count
Store adapters	Parsing of saved search pages for all six stores; sponsored listings skipped; blocked pages raise errors; proxy routing	23
Matching	Normalisation, spec guards, variant and model separation, accessory detection, fuzzy rescue	31
Application	Pages, progressive fragments, API shape, caching, store failure isolation, price history, alert lifecycle	13
Evaluation guard	Matcher precision and recall must stay $\geq 90\%$ on the labelled set	2
Total (all passing)		69

7.2 Matching accuracy

A dataset of 78 hand-labelled pairs of real listing titles was created (40 same product, 38 different: storage tiers, Pro/Plus/Ultra variants, generations, accessories, screen and case sizes, spelling variations). A pair is predicted "same" when the matcher places both listings in one row.

Matcher version	Precision	Recall	F1	Accuracy
Initial (Jaccard + basic spec guards)	78.6%	82.5%	80.5%	80.8%
Improved guards (model numbers, RAM+storage, screen tolerance)	92.5%	92.5%	92.5%	92.3%
+ fuzzy tiebreaker (current)	92.7%	95.0%	93.8%	93.6%

The evaluation exposed concrete defects that were then fixed: single-digit model numbers were being discarded (so "Flip 6" merged with "Flip 5"), RAM and storage only needed one shared value, four-digit model numbers were ignored, and "with MagSafe Case" marked a product as an accessory. The remaining five errors are cases where the only difference is a product-line word ("MX Master" vs "MX Anywhere", "Disc" vs "Digital Edition").

7.3 Live measurements

On the deployed copy, a search for "iphone 15" returned results from all stores: Amazon 12 results in 2.5 s, Flipkart 12 in 9.2 s, Snapdeal 12 in 2.4 s, Vijay Sales 9 in 0.4 s, Croma 12; these were grouped into 8 product rows. The spread between Flipkart and Vijay Sales motivated the progressive-loading design.

8. Deployment

The application is deployed on Vercel as a serverless FastAPI function and is also runnable locally (`make dev`, `make demo`) and with Docker. The source is version-controlled on GitHub with feature-

level commits. On Vercel only a temporary directory is writable, so the SQLite data (cache, history, alerts) does not survive function restarts; this is the main limitation of the current hosting and is addressed in the remaining work.

9. Challenges and Limitations

- **Bot protection.** Stores block data-centre IPs (HTTP 403/503, CAPTCHA). Solved for the hosted copy with a residential proxy service; Croma's API additionally required the storefront's channel code and percent-encoded queries, discovered by testing from the cloud.
- **Fragile markup.** Store HTML changes without notice. Mitigated by preferring embedded JSON and search APIs over CSS selectors, and by fixture-based tests that fail when parsing breaks.
- **Matching ambiguity.** Some products differ by one word only; the rule-based matcher cannot know that "Anywhere" and "Master" are different product lines. A learned classifier trained on the labelled pairs is a possible extension.
- **Serverless storage.** SQLite is not durable on Vercel.
- **Scope.** Prices exclude delivery charges and bank offers; colour variants are treated as the same product.

10. Work Completed and Remaining Work

Phase	Work	Status
1	Requirements, store analysis, architecture	Completed
2	Four store adapters, parallel search, cache, basic UI	Completed
3	Matching algorithm with spec/variant/model guards	Completed
4	Deployment on Vercel with proxy; GitHub repository	Completed
5	Progressive loading, price history, price-drop alerts, Croma and Reliance Digital, labelled evaluation set, UI redesign	Completed (midterm)
6	Durable hosted database (Postgres/Redis) so history and alerts persist in production	Planned
7	Continuous integration (run tests on every push) and automatic deployment from GitHub	Planned
8	Larger labelled dataset (200+ pairs) and a learned matcher compared with the rule-based one	Planned
9	User study with 10–15 participants; final report and presentation	Planned

11. Conclusion

At the midterm stage PriceCompare is a working, deployed application that meets the core objectives: it searches six stores in parallel, groups the same product across them with measured 93.6% accuracy, shows prices side by side with savings and a 30-day trend, and supports price-drop alerts. The engineering emphasis has been on resilience (isolated store failures, cached results, fixture tests) and on measurable quality (a labelled evaluation set with a regression guard). The remaining work focuses on durable storage for the hosted copy, automation of testing and deployment, and strengthening the matcher with more data.

References

1. FastAPI documentation. <https://fastapi.tiangolo.com/>
2. Beautiful Soup documentation. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
3. rapidfuzz: rapid fuzzy string matching. <https://github.com/rapidfuzz/RapidFuzz>
4. P. Jaccard, "The distribution of the flora in the alpine zone," *New Phytologist*, vol. 11, no. 2, pp. 37–50, 1912.
5. V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, pp. 707–710, 1966.
6. Vercel documentation: Python functions and Cron Jobs. <https://vercel.com/docs>
7. pytest documentation. <https://docs.pytest.org/>
8. Project repository. <https://github.com/alizeeshan1234/price-compare-capstone-project>